Authentication of administrative documents using blockchain technology

Zilèdem Pierre Canisius HIEN[1[0000-1111-2222-3333]], Ousmane BARRA[2[1111-2222-3333-4444]], Yaya TRAORE[2[1111-2222-3333-4444]], and Sadouanouan MALO[1[0000-1111-2222-3333]]

[1] Dep. of Computer Science, University NAZI BONI, Bobo-Dioulasso, BURKINA FASO, hiencanisius@gmail.com

[2] Dep. of Computer Science, University Joseph KI-ZERBO, Ouagadougou, BURKINA FASO

Abstract. Administrative documents have always played a central role not only in the functioning of the Administration but also in the relationship between the Administration and its users/customers. Indeed, they are used for traceability and serve as a basis for decision-making, communication and legal purposes. This means that all administrative documents must be reliable and of indisputable origin. However, it is increasingly common to encounter falsified digital administrative documents or those of dubious origin. This phenomenon creates a crisis of confidence between users/customers and the administration. Blockchain is a revolutionary technology can help secure administrative documents. In this paper, we propose a decentralized approach to authenticating administrative documents based on the non-repudiation, transparency and decentralized storage properties of blockchain technology. We combine the use of a smart contract, the SHA-256 (Secure Hash Algorithm 256-bit) hashing algorithm, digital signatures and electronic time stamping. Our process begins with the registration of administrative documents on the blockchain by an administrative entity. Thus, any user (or other administrative entity) wishing to verify the authenticity of an administrative document in their possession can also upload it to the Ethereum blockchain, which will (after research and processing) confirm or deny the authenticity of said document. This solution not only guarantees greater security and integrity for administrative documents, but also offers the ability to verify their authenticity in real time in a simple manner. Simulations show that it is potentially useful for administration.

Keywords: Authentication, document, blockchain, decentralized application, security.

1 Introduction

Document authentication is a process whereby a computer system or a human being proves or certifies that a document is authentic. A document is considered authentic if it is the original, or a copy that conforms to the original after verification and validation by an authorized or competent person. However, this form of certification is not intended to prove the veracity of the document's content. Its purpose is to ensure that the

signature and stamp visible on the document originate from an official authority and that the date of signature or issue, the name and position of the signatory are also legible.

The reliability of certain administrative documents is increasingly controversial, as it is now common to find falsified documents or documents of dubious origin in the public digital space, causing confusion among many citizens. This could be due in part to the development and misuse of multiple tools based, for example, on Artificial Intelligence (AI).

Indeed, the phenomenon of falsified documents contributes to tarnishing the image of their legitimate author, which could be the government, thus creating a crisis of confidence between the government and its users/customers. To address this phenomenon, authors and recipients of documents sometimes carry out authentication operations, at enormous cost in terms of time and money, which could ultimately reduce their returns.

Blockchain is a technological revolution that has been rapidly developing in recent years. Known for its non-repudiation, transparency and decentralized storage properties, it offers numerous advantages in many aspects of human life and is proving to be an effective means of improving government public services in particular. As a result, the use of this technology in the authentication process for administrative documents could bring greater ease, reliability and security.

In this paper, we propose an approach to document authentication using blockchain technology. Indeed, are therefore interested in the use of blockchain technology, particularly its consensus, verification and validation protocols, to resolve the issue of authenticating administrative documents, in order to reduce the risk of fraud and falsification of these documents.

The rest of the paper is organized as follows: in Section 2 we present a state-of-the-art review of existing work. In Section 3 we present our methodology. In Section 4 we present a simulation of the approach. We end with a conclusion and prospects in Section 5.

2 State of the art

Blockchain technology, propelled by the advent in 2008 of Satoshi Nakamoto's bitcoin cryptocurrency [1], has been growing rapidly in recent decades. It is a digital technology that allows information to be stored chronologically and transmitted in the form of blocks linked together securely (heavy use of cryptography) and without a central authority.

According to Imran Bashir [2], third-generation blockchains are applicable beyond the financial services industry in various general-purpose societal areas such as media, healthcare, government, etc. In addition to Bitcoin, there are several blockchains (Solana, Ethereum, Cardano, Avalanche, etc.), among which we have chosen Ethereum for this experiment.

2.1 Ethereum Blockchain

The public Ethereum blockchain, whose consensus mechanism switched from Proof of Work (PoW) to Proof of Stake (PoS) in September 2022 [3], underwent several changes after it was announced at the North American Bitcoin Conference in Miami in January 2014 and then launched on 30 July 2015 [4]. In the Ethereum Whitepaper published in 2013 [5], Vitalik Buterin stated that "*The intent of Ethereum is to create an alternative protocol for building decentralized applications, providing a different set of tradeoffs that we believe will be very useful for a large class of decentralized applications, with particular emphasis on situations where rapid development time, security for small and rarely used applications, and the ability of different applications to very efficiently interact, are important.*". In line with this dynamic, Ethereum 2.0 regularly introduces numerous developments, such as its May 7, 2025 update via the Pectra hard fork, which makes the network more secure, accessible, and user-friendly.

The ability to produce decentralized applications based on Ethereum – operating on a peer-to-peer network – relies heavily on the use of smart contracts. This makes Ethereum a Turing Complete blockchain [6], unlike the Bitcoin network. In certain use cases of Ethereum, such as ours, we must use two (02) accounts. The first is an **Externally Owned Account (EOA)** controlled by a cryptographic private key, with several hundred ETH and identifiable by an address (a 42-characters hexadecimal hash prefixed with 0x). The second is a **Contract Account (CA)** whose address is obtained after deployment (using ≈ 782016 Gas) of our smart contract.

Technically, a smart contract refers to a computer program implemented in a high-level programming language such as Solidity, Vyper, etc., whose code is compiled into bytecode and deployed on a specific version of the blockchain virtual machine – the EVM, in the case of Ethereum. It is designed to run autonomously when certain criteria are met. In addition to their advantages in terms of speed, accuracy, transparency and trust, multiple data backups, cost reduction, and security [7], smart contracts are, according to [2], a revolutionary feature of blockchain because they allow for flexibility, empowerment and highly desirable control over the actions desired by users.

2.2 Document authentication and security

According to Article 4 of [8], the following are considered administrative documents: "*documents produced or received, in the context of public service, by the State, local authorities and other public or private entities entrusted with such a mission;*". These include, for example, memos, decisions, instructions, circulars, directives, journals, deliberations, reports, minutes, sketches, plans, diagrams, notices, forecasts, official announcements, certificates (of commencement, resumption, or termination of service, etc.), bulletins, decrees, orders, etc.

Given that a document refers to information stored on paper or electronic media, document authentication is a process whereby a computer system or human being proves or certifies that a document is authentic. In general, for authorized individuals – civil registrars, police officers, official authorities that issued the document, etc. – the

process consists of ensuring that the signature and stamp visible on the document (original, certified copy of the original) originate from an official authority and that the date of signature or issue, the name and position of the signatory are also legible.

In all cases, authentication is a component of security, which itself is a set of measures to be taken and implemented to ensure traceability related to access and the protection of sensitive information (electronic or physical).

2.3 Existing methods of document authentication

The work of Omar S. Saleh et al [9] focused on a blockchain-based framework for verifying educational certificates. This framework is proposed using the Hyperledger Fabric Framework. The authors identified and studied the security issues (authentication, authorization, confidentiality, privacy, and ownership) necessary for verifying documents through blockchain. This study reveals that verifying these types of documents is feasible but remains a fairly complex task involving various tedious authentication processes. This is because in legitimate documents, the most frequently falsified parts are the dates of enrollment/graduation, grades, course content, date of birth, specialization, etc.

In [10], a unique document verification model was proposed. This model, typical of the public sector, is designed to resolve the tedious and time-consuming process of document verification by exploring the efficiency and security that blockchain could bring. Indeed, as the objective of the study was to develop a private, authorized blockchain network where citizens' official documents could be shared by government agencies and educational institutions, the author set up a decentralized system that minimizes the level of human intervention and increases security, reliability, and transparency.

A system model called SECHash (Scanning, Extraction, Confirmation, and Hashing of Documents) was proposed by [11]. This system integrates blockchain and optical character recognition (OCR) technologies with the aim of not only maintaining the integrity and accuracy of incoming invoice data by prohibiting modification after acceptance by government agencies, but also ensuring that accepted invoices are neither destroyed nor lost. SECHash was implemented using LBRY as the public blockchain network, Django as the web framework, PostgreSQL as the metadata and application data database, and Aspire for OCR. This model provides a good foundation for electronic document management using blockchain technology in e-governance. However, the author acknowledges that the use of LBRY and Aspire does not seem generic enough to apply this model in a standard way to multiple diverse domains. In addition, the good faith of government actors would be very necessary to significantly reduce electronic document fraud.

The transition from traditional administration to modern administration using technology, particularly blockchain, would strengthen the stability and integrity of information while preventing corruption. Indeed, studies conducted by Rana F. Ghani et al. [12] show that the main advantages of applying blockchain technology to electronic administrative systems include: the ability to store and share government, citizen, and business data in a secure and controlled manner; reduced management costs and risks

of corruption/abuse; the elimination of human intermediaries; and increased trust among users of e-government services. With this in mind, the authors [12] developed a computer system (FBCOLM) using Java TCP socket and blockchain technology, which has shown promising results in testing. This system, designed for managing official letters, takes into account the addition of new official documents and the verification of existing documents.

Isyak Meirobie et al. [13] have set up a framework for authenticating electronic documents using Blockchain technology for the Indonesian government system. Go-Chain stores the hashed (in SHA-256) signature (P-256 curve DSA) of the digital transcript of the document on a public blockchain. This transcript is also distributed to the public in json format. The author opted for the Proof of Authority (PoA) consensus protocol.

Ana BAKHOUM [14] has also proposed a decentralized system for backing up and securing electronic school reports in the Ethereum blockchain via a smart contract. This aims to ensure the reliability, authenticity, transparency and security of electronic reports.

Our solution innovates with the integration of electronic timestamping and the non-direct manipulation of cryptographic keys by the general public. It offers the possibility of verifying the authenticity of documents in real time, securely, transparently, and without going through a central verification authority.

3 Methodology

Our authentication approach (see Fig. 1) consists of two main phases that are ordered and complementary.

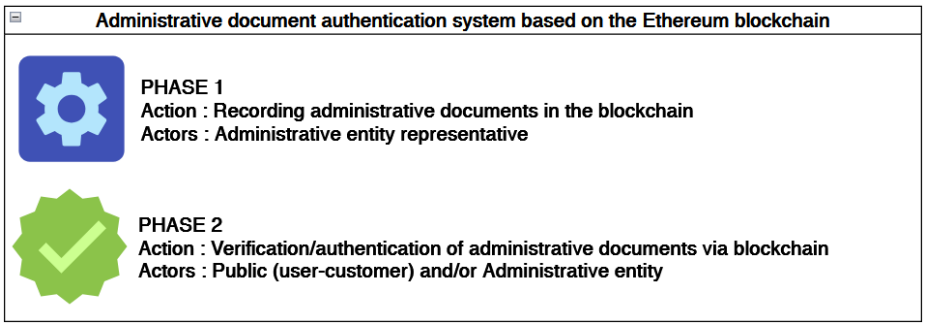

Fig. 1. Constitutive phases of the approach to authenticating administrative documents using the Ethereum blockchain.

Each of these phases consists of several stages.

3.1 Phase 1 - Recording administrative documents in the Ethereum blockchain

The execution of Phase 1 (see Fig. 2) requires the payment of gas fees.

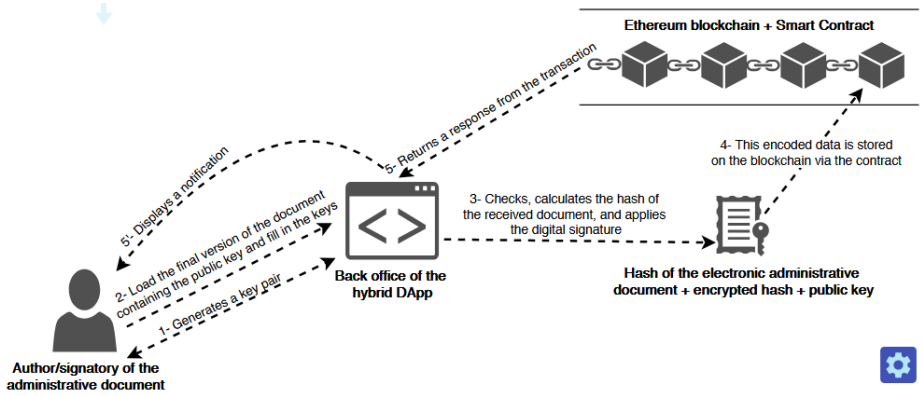

Fig. 2. Phase 1 - Recording administrative documents in the Ethereum blockchain.

The process for this Phase 1 is described as follows:

- **1-Generates a key pair**: the author generates a pair of asymmetric keys through the Decentralized Application (DApp) back office. These keys must not be lost, and the private key must be carefully stored.
- **2-Load the final version of the document containing the public key and fill in the keys**: the author integrates their public key into the document so that it is visible. However, this remains optional. They then upload (via the DApp back office) the final PDF or Word version of the document containing their public key. During the upload, they enter the key pair. The front end transmits this data to the DApp back end.
- **3-Checks, calculates the hash of the received document, and applies the digital signature**: the backend extracts the (text) content of the uploaded file and calculates a hash (or digital fingerprint) of this content. The backend then encrypts the calculated hash with the author's private key. This produces a signed hash, which is the document's digital signature. This data (the raw hash, the signed hash, and the author's public key) is encoded and sent back to the DApp frontend.
- **4-This encoded data is stored on the blockchain via the contract**: the frontend then transfers the encoded data received from the backend (raw hash, signed hash, and public key) to the smart contract – by calling the storeAdministrativeDocument() method–, which stores it on the Ethereum 2.0 blockchain. During storage, the timestamp is calculated and associated with the data by the smart contract. Thus, when a user wants to verify authenticity, the corresponding public key is used to decrypt and verify the document's signature.
- **5-Returns a response from the transaction**: a notification from the blockchain transaction is returned to the frontend. (5'-Displays a notification) This notification is customized and displayed to the user.

3.2 Phase 2 - Verification or authentication of administrative documents on the Ethereum blockchain

This phase is illustrated in Fig. 3. No fees are payable here as it is simply a matter of reading data in Ethereum. This seems to justify Jean-Paul Delahaye's [15] expression *"(...) freely and without charge, everyone can read (...)"* in his definition of blockchain.

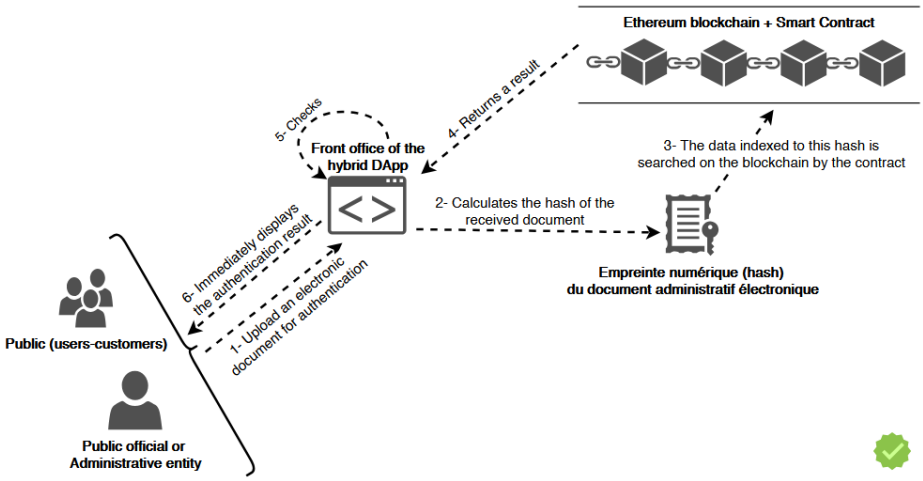

Fig. 3. Phase 2 - Verification or authentication of administrative documents on the Ethereum blockchain.

Phase 2 also consists of several steps that can be described as follows:

- **1-Upload an electronic document for authentication**: the user (public or public official) uploads the PDF or Word document through the DApp front office whose authenticity they wish to verify. The frontend transmits this data to the DApp backend.
- **2-Calculates the hash of the received document**: the backend extracts the (text) content of the uploaded file for authentication and calculates the hash of this content. This digital fingerprint (hash) is encoded and sent back to the frontend of the Dapp.
- **3-The data indexed to this hash is searched on the blockchain by the contract**: the frontend uses this hash and queries the Ethereum 2.0 blockchain via the smart contract (deployed there) to retrieve the raw digital fingerprint, digital signature, and public key previously stored in the blockchain (through phase 1). This search is possible by using the getAdministrativeDocument() method of the contract.
- **4-Returns a result**: the blockchain returns a response to the DApp via the smart contract. This response is then transmitted to the DApp's backend for verification.
- **5-Checks**: the backend decodes the data received from Ethereum and performs checks: if an occurrence is found (potentially intact document), the

DApp verifies the digital signature associated with this occurrence via the previously stored public key. If this verification fails, then the signature does not come from the correct signatory, i.e., the author who validated the final version of the document. This is the verification of the document's authenticity. If the signature is valid, then the document is declared authenticated and authentic.

- **6-Immediately displays the authentication result**: a notification is sent to the frontend. This means that the authentication result is immediately personalized and displayed to the user.

3.3 Implementation architecture

Integrating smart contracts into a traditional application allows interaction with the blockchain. For successful integration, we have adopted a clear DApp architecture (see Fig. 4) where the responsibilities of each component are well defined.

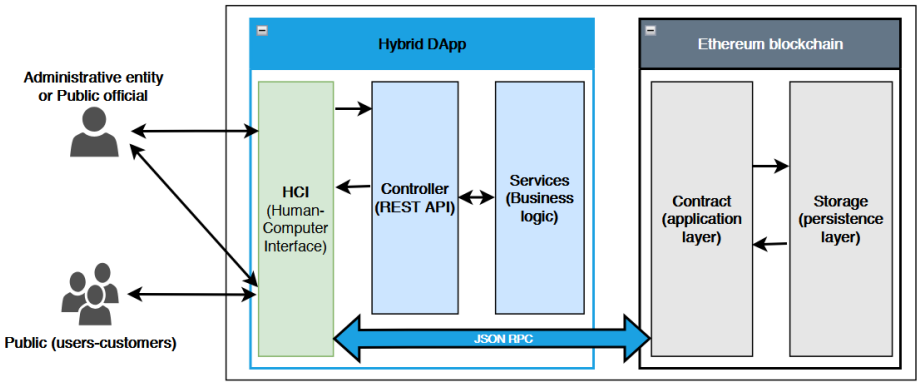

Fig. 4. Simplified layered architecture of the solution.

The Human Computer Interfaces (HCI or frontend) layer consists of two areas: a back office that is only accessible by the administrative entity, and a front office intended for the general public. The Controller layer, which is the upper web layer of the backend, is responsible for exposing REST APIs for the benefit of the HCI layer. These APIs provide resources (hash, signature, etc.) that are calculated by the Services layer (responsible for business logic). The responsibility of the application layer of the Ethereum blockchain is to host our smart contract and to work with the other layers of this blockchain to execute the contract. This application layer communicates with the HCI layer. It also communicates with the Ethereum persistence layer, where the hashed, encrypted, and encoded data from administrative documents is stored and searched. This is what enables decentralized storage of these documents through immutable transaction requests. The various interactions (transaction requests) between the EOA account and our smart contract are shown in Fig. 5.

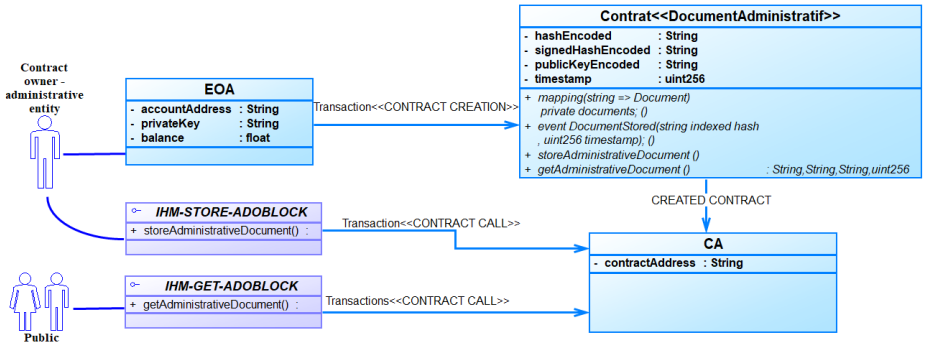

Fig. 5. Diagram of interactions between the smart contract and accounts.

In addition to this layered architecture and clarification of the interactions between the various components of the solution, the integration of this decentralized application into the current government system remains an important issue to be resolved. This is because the administration's network does not operate on a peer-to-peer basis like blockchain, particularly Ethereum. To adequately address this concern, our future work will focus on the deployment architecture of this solution in order to implement it in a historically centralized storage and networking system.

4 Testing the solution

4.1 Tools

The implementation requires at least one computer with a Core i7 processor, 16 GB of RAM, and 128 GB of SSD storage, with no operating system requirements.

The backend and frontend parts of the DApp were implemented with Visual Studio Code 1.96.2 using the Spring-boot 3.4.1 (Maven 3.8.1 + Java 17) and Angular 15 (HTML5, TypeScript/JavaScript, SCSS/CSS3) frameworks.

The smart contract was implemented and tested in Solidity ^0.8.28 on Remix-Ethereum IDE. The MetaMask v2.132.0 cryptocurrency wallet was integrated for blockchain transaction requests and signatures, which are made via the JSON Remote Procedure Call (JSON RPC) protocol to Angular via the Application Binary Interface (ABI). In terms of core technologies, we experimented with the Hardhat ^2.24.0 framework and the Ethers.js ^6.14.0 library to configure the test blockchain network (Localhost network), compile, and deploy our contract. In order to generate ECDSA (Elliptic Curve Digital Signature Algorithm) cryptographic key pairs, we use the Maven bouncycastle dependency; this allows us to use the secp256r1 (P-256) elliptic curve for digital signatures of digital fingerprints calculated using the SHA-256 algorithm. P-256 is compatible with Ethereum and the standards of the National Institute of Standards and Technology (NIST) [16] [17].

4.2 Simulations

According to our experiments on official announcements, storing a file of approximately 500 KB requires an average of 322,200 Gas, or 389,212.119378 Gwei (0.000389212119378 ETH, given that 1 ETH = 10^9 Gwei = 10^{18} Wei). ETH refers to "Ether", the native cryptocurrency of the Ethereum blockchain. The transaction to verify the authenticity of a document does not incur any fees.

The Table 1 below presents the results of an experimental simulation. This test is realistic (with manual transaction confirmation/signature) and is not considered a load test. In our context, the load test would be limited by the fact that human intervention is required by MetaMask. According to the summary table, **Tbs** (Time before signing in milliseconds – ms) refers to ADOBLOCK's response time before the user explicitly signs to authorize the transaction fees from their wallet. If the document does not exist during authentication, if the document exists during registration, or if there is any error, processing also ends at this time. **Tas** (Time after signing in milliseconds) is ADOBLOCK's response time after the user explicitly signs to authorize the transaction fees to be debited from their wallet. If the transaction is successful, processing ends at this time. This means that Tbs and Tas are the cumulative interaction time between the user, frontend, Ethereum, and backend components.

Table 1. Summary of simulations.

Test	Doc file (Ko)	Tbs (ms)	Tas (ms)	Cost (ETH)
1st-call to authenticate administrative documents (does not exist)	110	2.10		0
	111	2.90		0
	111	1.10		0
	111	1.30		0
	28	1.40		0
	Avg: 94.2	Avg: 1.76	Avg: 0	Avg: 0
	Min: 28	Min: 1.10	Min: 0	Min: 0
	Max: 111	Max: 2.90	Max: 0	Max: 0
2nd-call to store administrative documents, i.e. user signature + eth consensus (new doc)	110	2.30	7421.10	0.0005705967265542
	111	1.70	4332.90	0.0005402140810758
	111	1.40	3640.80	0.0005135476887864
	111	2.00	2892.20	0.0004901429963700
	28	1.10	2951.30	0.0004696010490204
	Avg: 94.2	Avg: 1.7	Avg: 4247.66	Avg: 0.0005168205083614
	Min: 28	Min: 1.10	Min: 2892.2	Min: 0.0004696010490204
	Max: 111	Max: 2.30	Max: 7421.1	Max: 0.0005705967265542
3rd-call to store administrative documents (doc already exists)	110	1.10		0
	111	1.10		0
	111	1.40		0
	111	1.20		0
	28	1.30		0
	Avg: 94.2	Avg: 1.22	Avg: 0	Avg: 0

	Min: 28	Min: 1.10	Min: 0	Min: 0
	Max: 111	Max: 1.40	Max: 0	Max: 0
4th-call to authenticate administrative documents (exists)	110	1.70	268.70	0
	111	2.30	117.20	0
	111	1.30	81.30	0
	111	1.60	115.50	0
	28	1.30	97.90	0
	Avg: 94.2	Avg: 1.64	Avg: 136.12	Avg: 0
	Min: 28	Min: 1.30	Min: 81.3	Min: 0
	Max: 111	Max: 2.30	Max: 268.7	Max: 0

Ultimately, for administrative documents averaging approximately 94.2 KB, the following would be required on average:

- 1.76 millisecond for an authentication attempt (free of charge) of a document that is not previously registered on Ethereum;
- 4247.66 milliseconds (4.24 seconds) and 0.0005168205083614 ETH to register the document on Ethereum;
- 136.12 milliseconds (0.13 second) for authentication (free of charge) of the administrative document.

Compared to the manual administrative document authentication system, our solution allows for real-time or near-instantaneous authentication.

The solution we propose, under the name ADOBLOCK, is accessible through a web browser. Fig. 6, Fig. 7, and Fig. 8 show the main user interfaces.

Fig. 6. Get private/public keys.

This interface in Fig. 6 allows you to obtain Elliptic Curve Digital Signature Algorithm (ECDSA) type key pairs and save them in a compressed file. These keys are reusable as long as the private key does not appear to be compromised.

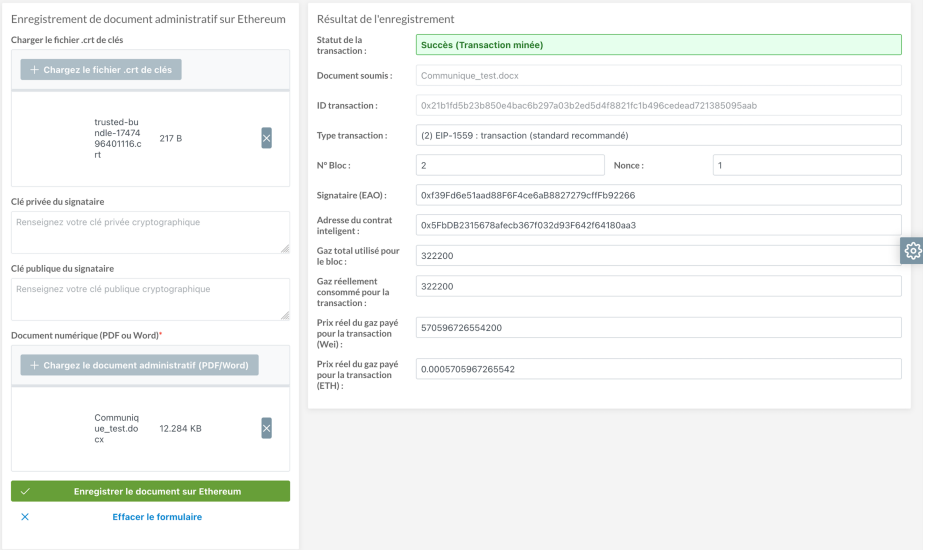

Fig. 7. Store administrative documents.

The Fig. 7 shows the recording of an administrative document in the blockchain. In addition to the status, hash (ID) and type of the Ethereum transaction, nonce, block number, signer (EOA) and smart contract addresses, etc., the solution displays the amounts of gas used and fees paid.

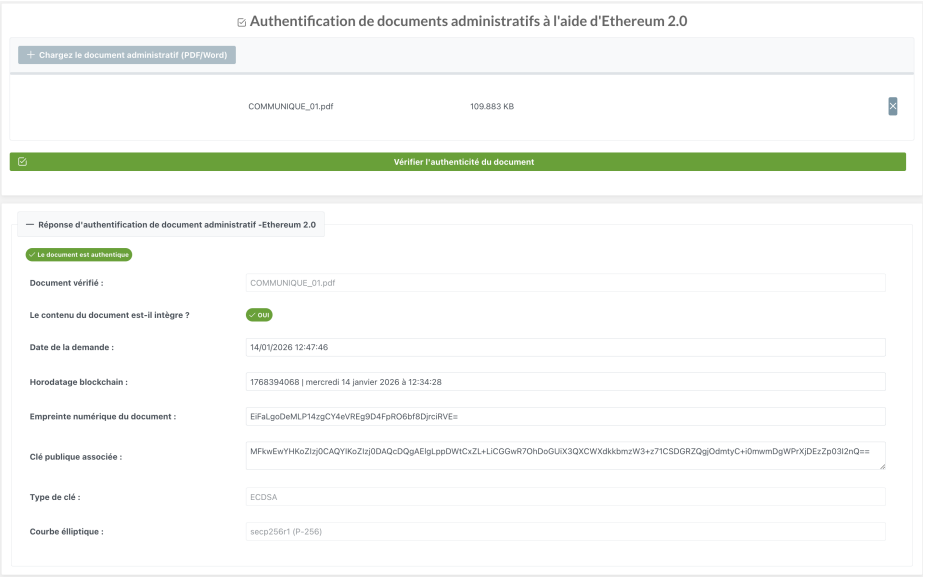

Fig. 8. Verify or authenticate document.

This interface displays the result of an authentication operation. It includes the timestamp, the integrity status of the document submitted for authentication, and the public key of the document's author found on the blockchain. This timestamp, initially in time format (e.g., 1749165548), is formatted as a complete date (e.g., lundi 09 juin 2025 à 09:59:08) that is easily readable for the end user. The decoded digital fingerprint, key types, and signature curve are also displayed.

5 Conclusion

Faced with the rise of document falsification, it is crucial to leverage technological advances and innovations to rethink methods of authenticating and securing digital documents. To this end, we have proposed an approach to document authentication using blockchain technology. To test this approach, we have implemented a hybrid decentralized application that interacts with the Ethereum blockchain using a smart contract. In this solution, we have integrated the use of electronic timestamping, digital signatures based on asymmetric keys, and hashing techniques using the SHA-256 algorithm.

The results of our experiments confirm that blockchain technology shows promise for guaranteeing the authenticity and security of administrative documents. However, further consideration should be given to the sovereignty of digital data and the integration of other pipelines to support electronic documents with non-textual content. Therefore, we will need to address the concern of how our system will function in a non-peer-to-peer network such as that of the current administration, and how to adapt the public expenditure system to take into account gas fee payments.

References

1. R. Subramanian et T. Chino: The State of Cryptocurrencies, Their Issues and Policy Interactions. J. Int. Technol. Inf. Manag., vol. 24, no 3, p. 40, (2015), doi: 10.58729/1941-6679.1045.
2. I. Bashir: Mastering Blockchain. Packt Publishing Ltd, (2017).
3. G. A. Pierro et A. Amoordon: Gas Fees and Unconfirmed Transactions in Ethereum: A Proof-of-Stake (PoS) Focus. in 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion (SANER-C), 1-8, (2024), doi: 10.1109/SANER-C62648.2024.00011.
4. D. Majalikar, S. Sudhir, et D. S. Abhang: BLOCKCHAN USING ETHEREUM – THE NEXT COMPUTING PARADIAGM SHIFT. Navajyoti Int. J. Multi-Discip. Res., vol. 3, no 2, p. 6, (2019).
5. B. Vitalik, et al.: Ethereum white paper. GitHub repository, vol. 1, no 22-23, 5-7, (2013).
6. S. S. Kushwaha, S. Joshi, D. Singh, M. Kaur, et H.-N. Lee: Systematic Review of Security Vulnerabilities in Ethereum Blockchain Smart Contract. IEEE Access, vol. 10, p. 6605-6621, (2022), doi: 10.1109/ACCESS.2021.3140091.
7. M. Krichen: Les méthodes formelles pour la vérification des contrats intelligents. https://hal.science/hal-03753067, last accessed 2025/11/19.
8. BURKINA FASO: Loi N°051-2025/CNT du 30 août 2015 portant droit d'accès à l'information publique et aux documents administratifs. https://www.csc.bf/index.php/textes-

dereference/lois/item/76-loi-051-portant-sur-l-acces-a-l-information-publique, last accessed 2024/09/16, p.4.

9. S. S. OMAR, G. OSMAN, et E. R. MUHAMMAD: BLOCKCHAIN BASED FRAMEWORK FOR EDUCATIONAL CERTIFICATES VERIFICATION. J. Crit. Rev., vol. 7, no 03, p. 6, (2020), doi: 10.31838/jcr.07.03.13.
10. G. Malik, K. Parasrampuria, S. P. Reddy, et S. Shah: Blockchain Based Identity Verification Model. In 2019 International Conference on Vision Towards Emerging Trends in Communication and Networking (ViTECoN), p. 1-6, (2019), doi: 10.1109/ViTECoN.2019.8899569.
11. F. Azzam et al.: The Use of Blockchain Technology and OCR in E-Government for Document Management: Inbound Invoice Management as an Example. Appl. Sci., vol. 13, no 14, p. 8463, (2023), doi: 10.3390/app13148463.
12. R. F. Ghani, A. A. S. Al-Karkhi, et S. M. Mahdi: Proposed Framework for Official Document Sharing and Verification in E-government Environment Based on Blockchain Technology. Baghdad Sci. J., vol. 19, no 6(Suppl.), p. 1592, (2022), doi: 10.21123/bsj.2022.7513.
13. I. Meirobie, A. P. Irawan, H. T. Sukmana, D. P. Lazirkha, et N. P. L. Santoso: Framework Authentication e-document using Blockchain Technology on the Government system. Int. J. Artif. Intell. Res., vol. 6, no 2, Art. no 2, (2022), doi: 10.29099/ijair.v6i2.294.
14. A. Bakhoum: La Blockchain pour la Sécurisation des E-livrets scolaires. http://rivieres-dusud.uasz.sn/xmlui/handle/123456789/1803, last accessed 2024/09/10, Location (2019).
15. T. Boudès: La blockchain déchaîne les questions! Ann. Mines - Gérer Compr., vol. 131, no 1, p. 83-85, (2018), doi: 10.3917/geco1.131.0083.
16. É. (gapz) Gaspar: Standardisation des courbes elliptiques : à qui faire confiance ? | Connect - Editions Diamond. https://connect.ed-diamond.com/MISC/mischs-013/standardisation-des-courbes-elliptiques-a-qui-faire-confiance, last accessed 2025/04/18, (2016).
17. C. de la sécurité des télécommunications Canada: Conseils sur la configuration sécurisée des protocoles réseau (ITSP.40.062). https://www.cyber.gc.ca/fr/orientation/conseils-sur-la-configuration-securisee-des-protocoles-reseau-itsp40062, last accessed 2025/04/18, (2020).